

Python 핵심 문법 학습 노트

인덴트(들여쓰기) 규칙부터 객체 지향 프로그래밍, 예외 처리까지 완벽 요약

1. 파이썬의 특징 및 변수 선언

파이썬은 인터프리터 언어이자 동적 타이핑(Dynamic Typing)을 지원하여 변수 선언 시 타입을 명시하지 않고, 대입되는 값에 따라 타입이 자동으로 결정됩니다.

📌 코드 블록과 들여쓰기 규칙

파이썬은 타 언어와 달리 중괄호 `{ }`를 사용하지 않고, **들여쓰기(Indent - 공백 4칸 권장)** 자체로 코드 블록의 범위를 규정하므로 일관된 들여쓰기가 문법적으로 매우 중요합니다.

```
# 변수 선언 및 동적 타이핑 예시
age = 25          # Integer
pi = 3.14159      # Float
name = "Python"   # String
is_active = True  # Boolean (대문자 주의)
```

2. 핵심 자료구조 (Data Structures)

파이썬은 데이터를 효과적으로 관리할 수 있는 강력한 내장 자료구조 컬렉션을 제공합니다.

자료형	특징 및 문법	예시 코드
리스트 (List)	순서가 있고 수정 가능한 배열 구조 <code>[]</code>	<pre>fruits = ["apple", "banana"] fruits.append("cherry")</pre>
튜플 (Tuple)	순서가 있지만 수정이 불가능 (Immutable)한 구조 <code>()</code>	<pre>point = (10, 20) # 값 변경 시 TypeError 발생</pre>
딕셔너리 (Dict)	Key-Value 쌍으로 이루어진 해시 맵 구조 <code>{ }</code>	<pre>user = {"name": "Alice", "age": 25} print(user["name"])</pre>
집합 (Set)	순서가 없고 중복을 허용하지 않는 구조 <code>{ }</code>	<pre>nums = {1, 2, 2, 3} # 결과는 {1, 2, 3}</pre>

3. 제어문 및 컴프리헨션 (Comprehension)

조건문 및 반복문

조건문에는 `elif`를 사용하며, 반복문은 `for ... in` 문법을 주로 사용해 컬렉션을 순회합니다.

```
# 조건문 예시
score = 85
if score >= 90:
    print("A학점")
elif score >= 80:
    print("B학점")
else:
    print("C학점")

# 반복문과 range() 함수
for i in range(1, 4): # 1부터 3까지 반복
    print(f"{i}번째 반복")
```

리스트 컴프리헨션 (List Comprehension)

파이썬의 독특하고 강력한 문법으로, 기존 리스트를 바탕으로 새로운 리스트를 더 직관적이고 간결하게 생성할 수 있습니다.

```
# 일반적인 방식
squares = []
for x in range(1, 6):
    squares.append(x**2)

# 리스트 컴프리헨션 방식 (한 줄로 처리)
squares_comp = [x**2 for x in range(1, 6)] # [1, 4, 9, 16, 25]
# 조건식을 포함한 컴프리헨션
even_squares = [x**2 for x in range(1, 6) if x % 2 == 0] # [4, 16]
```

4. 함수와 가변 인자

함수는 `def` 키워드로 정의하며, 인자의 개수가 정해지지 않았을 때 가변 인자(`*args`, `**kwargs`)를 사용할 수 있습니다.

```
# 가변 인자를 사용하는 함수 정의
def profile_summary(*languages, **info):
    print(f"이름: {info.get('name')}")
    print("사용 가능 언어:")
    for lang in languages:
        print(f"- {lang}")

# 함수 호출
profile_summary("Python", "Java", "C++", name="Alice", age=30)
```

5. 객체 지향 프로그래밍 (OOP) 및 예외 처리

파이썬은 클래스 기반의 객체 지향 프로그래밍을 지원합니다. 생성자는 `__init__` 메서드를 사용하며, 모든 메서드의 첫 번째 인자는 인스턴스 자신을 가리키는 `self` 여야 합니다.

```
# 클래스 선언과 상속
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        pass

class Dog(Animal): # Animal 클래스 상속
    def speak(self):
        return f"{self.name}가 멍멍 짭니다."

my_dog = Dog("초코")
print(my_dog.speak()) # 초코가 멍멍 짭니다.
```

예외 처리 (Exception Handling)

프로그램 실행 중 예외가 발생했을 때 시스템이 멈추는 것을 방지하기 위해 **try-except** 블록을 활용합니다.

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"예외 발생: {e}") # 예외 발생: division by zero
else:
    print("오류 없이 성공적으로 실행되었습니다.")
finally:
    print("오류 여부와 상관없이 항상 실행되는 블록입니다.")
```